Building REST APIs with Spring Boot: Encapsulation, Jackson Serialization, and Postman Fundamentals

Learning Outcomes

- **Understand the role of encapsulation in Spring Boot**
  Explain why private fields with getters are required in Spring Boot.
- **Implement request mapping annotations**
  Correctly apply @PathVariable, @RequestBody and @RequestMapping annotations.
- **Utilize constructors effectively in Spring Boot**
  Differentiate between no-arg constructors and full-arg constructors for object creation.
- **Test REST API endpoints using Postman**
  Configure postman collections, environment variables, and send requests to test API endpoints.



What is JSON ?

```
{
  "id": 1,
  "name": "Laptop",
  "price": 999.99
}
```

Reading it:

{ } wraps one object

"key" : value is one field of that object

text is quoted, numbers are bare, fields are comma-separated

JavaScript Object Notation

A plain-text way of writing down structured data. Despite the name it does not run anything and is not tied to JavaScript - it is simply an agreed layout for values.

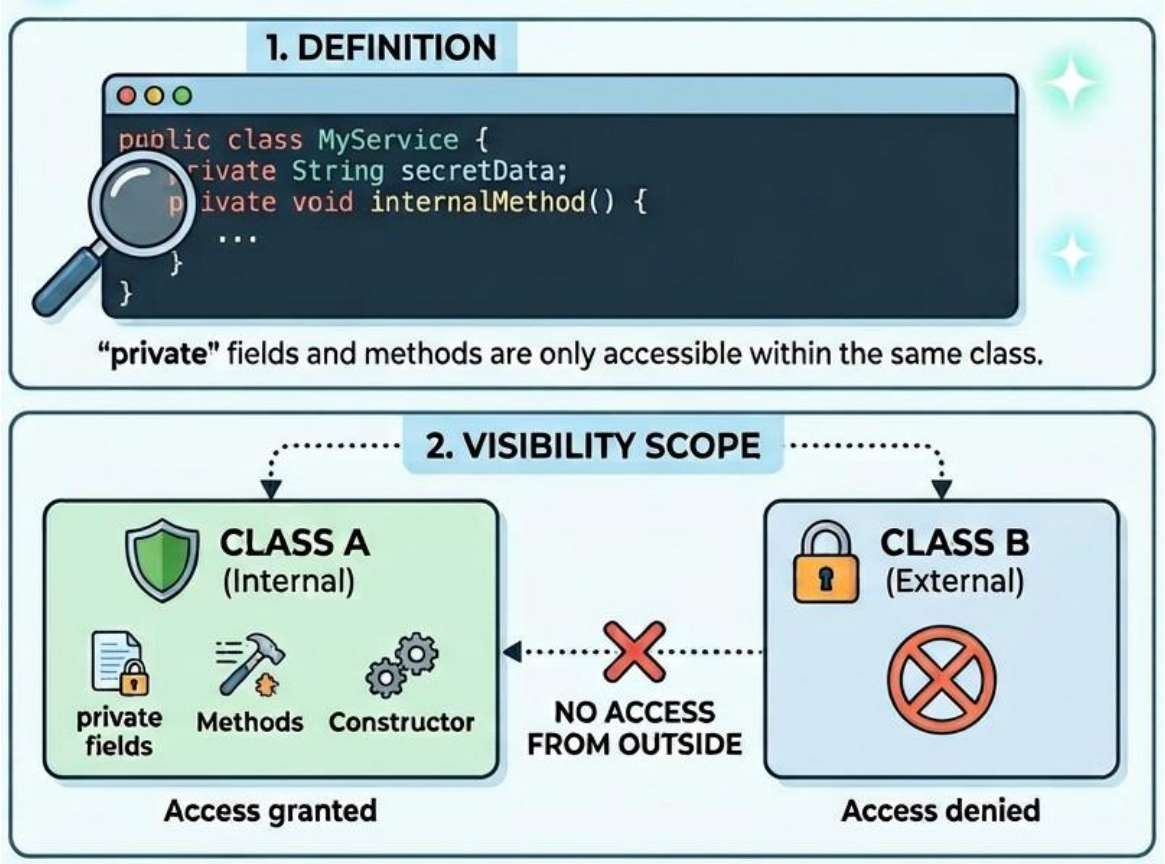
Why JSON rather than Java

The caller might be a React page, a Python script, an Android app or Swagger UI. JSON is language-neutral, so every one of them can read the same response.

The field names come from your Java class - but the format itself belongs to the whole web, not to Java.

Why Private Fields Matter in Spring Boot

Breaking the API on purpose



Public Fields Work Fine in Spring Boot

```
public class Product {
    public Long id;
    public String name;
    public Double price;
    ...
}
```

```
GET /products/1

{
  "id": 1,
  "name": "Laptop",
  "price": 999.99
}
```

✅ The JSON looks perfect. So, what's the problem?

The problem is public fields let any external code to directly modify an object's properties without the object's knowledge or permission.

Understanding the `@PathVariable` - Extracting Dynamic URL Values

URL Identifier Extraction

Extracts dynamic values directly from placeholder variables in the request URL path.

Single Object Identification

Used to target specific object, e.g., getting a product by unique ID.

Automatic Type Binding

Spring converts URL segments automatically into target types like Long or String.

```
// Controller Method     GET /products/{id}

@GetMapping("/products/{id}")
public Product getProductById(@PathVariable Long id ) {
      // add the required logic here ...
}
```

PATH MATCHING BREAKDOWN

GET /products/ 1 → id = 1L

💡 **Quick Distinction:** Use **@PathVariable** for single object identifiers (/products/1)

Breaking the API on purpose

```java
public class Product {
    private Long id;
    private String name;
    private double price;

    public Product() {}
    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
    // no getters yet...
}
```

`GET /products/1`

`{ }`

Empty JSON. No error thrown - just... nothing.

WHY THIS MATTERS

This is the clearest possible demonstration that encapsulation isn't just a style rule - it has real, visible consequences in a running Spring application.

The Diagnosis: Jackson Needs Getters



Jackson

- Spring uses a library called **Jackson** to convert your Java objects into JSON.
- Jackson does not read private fields directly - it looks for a public `getX()` (getter) method for each field it wants to **serialize.**
- **No getter → Jackson finds nothing to call → that field is silently left out of the JSON.**

What is Serialization Here: Java Object to JSON

1

Your method returns an object

```
new Product(1L, "Laptop", 999.99)
```

a real object, held in memory

2

Spring writes it out as text

```
"name" : "Laptop"
```

done for you by a library that came with starter-web

3

The client receives characters

```
{ "id": 1, "name": ... }
```

carried in the HTTP response body

The same machinery runs in reverse - deserialization: JSON arriving in a POST request body is turned back into a Product object.

The Fix: Add the Getter methods

❌ **BEFORE**

```
private String name;
// no getter
```

Result: "name" missing from every response

✅ **AFTER**

```
private String name;
public String getName() {
    return name;
}
```

Result: "name" appears correctly in the JSON

💡 **Core Rule:** Private fields protect internal state while public getters selectively expose API properties. Encapsulation directly dictates Spring's JSON output.

KEY TAKEAWAY

Encapsulation is not just a rule. It has real consequences.

private fields protect your object's state - but Spring's JSON serialiser can only expose what a **public getter** reveals. Design your class API deliberately: private data, public behaviour.

Constructors:
Full-Arg Constructor vs No-Arg Constructor

Why Spring quietly needs both constructors

Understanding the @RequestBody - Mapping JSON data to Java Object

Payload Binding

Binds the incoming HTTP request body directly into a Java object parameter.

Jackson Integration

Uses Jackson to convert JSON keys into matching Java fields via constructors and setters.

```
// Controller Method                    POST /products

@PostMapping("/products")
public Product saveProduct(@RequestBody Product product ) {
        // add the required logic here ...
}
```

INCOMING JSON PAYLOAD DATA

```
{
    "id": 1,
    "name": "Gaming Keyboard",
    "price": 89.99
}
```

⚠ **Important Requirement:** Jackson requires a **no-arg constructor** in Product class to deserialize JSON correctly.

Why Spring Often Needs a No-Arg Constructor

1

JSON arrives

```
{
  "name": "Laptop",
  "price": 999.99
}
```

as a @RequestBody

2

Jackson builds it

`new Product()`

then calls each setter / assigns each field

3

Without a no-arg constructor...

`new Product() has no matching constructor`

→ *deserialisation fails at runtime*

@RequestBody Fails Without It

```
// Product with ONLY the full-arg constructor:
public Product(Long id, String name, double price) {
    this.id = id;
    this.name = name;
    this.price = price;
}

@PostMapping
public Product saveProduct(@RequestBody Product product) {
        System.out.println(product);
        return product;
}

// POST /products
{"id":1, "name": "Mouse", "price": 19.99 }

// ❌ HTTP 400 — Jackson cannot construct Product
```

THE FIX

public Product() {}

Add this no-arg constructor alongside the full one - one line restores the endpoint.

Restart. Retry the POST. Watch it succeed.

The Pattern: Two Constructors, Two Purposes

```
public class Product {
    private Long id;
    private String name;
    private double price;

    public Product() {}                                          // for frameworks (Jackson)
    public Product(Long id, String name, double price) {    // for your own code
        this.id = id; this.name = name; this.price = price;
    }
}
```

No-arg constructor

Exists purely so frameworks can instantiate the object before populating it field by field.

Full constructor

What you actually call in your own code - guarantees every field is set at creation time.

Java Bean Convention

```
public class Product {
    private Long id;
    private String name;
    private Double price;

    public Product() {} // required by Jackson
    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public void setId(Long id) { this.id = id; }
    public Long getId() { return id; }

    public void setName(String name) { this.name = name; }
    public String getName() { return name; }

    public void setPrice(Double price) { this.price = price}
    public Double getPrice() { return price; }
```

Building an Immutable Product

```
public class Product {
    private final Long id = 1L;
    private final String name = "Laptop";
    private final double price = 999.99;

    public Product() {} // required by Jackson

    public Long getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    // no setters, properties are fixed for the lifetime of the object
```

Quick Check: Before We Move On …

Q1 **Why does Product need a no-arg constructor if it already has a full one?**

→ *Jackson uses it to construct the object before setting fields during deserialisation.*

Q2 **If a field has a getter but no setter, can it still change value?**

→ *Yes - inside the class itself. Outside code just can't change it directly.*

Q3 **What HTTP response would you expect from a private field with no getter?**

→ *It's simply omitted from the JSON - no error, no warning.*

Understanding the @RequestMapping Annotation

Class-Level Base Routing

Defines base URL paths (/products) for all controller endpoints.

Method-Level Specialized Shortcuts

@GetMapping, @PostMapping, and @DeleteMapping are shortcut annotations.

ProductController.java

```java
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}") // Maps GET /products/{id}
    public Product getProductById(@PathVariable Long id) {
            // add the required logic here …
    }
}
```

REST URL Rules and HTTP Verbs

🚫 Golden URL Rule: Nouns Only (Best Practice)

URLs should **only contain nouns**. Avoid putting verbs inside URL paths (e.g., use GET /products instead of GET /getAllProducts).

GET: Retrieve data (read, search, load).

POST: Create a resource in backend. *If resource already exists, return Error.*

DELETE: Remove a resource permanently.

PUT: Update or replace a resource. *If resource does NOT exist, create a new one.*

PATCH: Partially update resource. *If resource does NOT exist, return Error.*

What a Request Asks For: the Verbs

GET

`@GetMapping`

Read something. Changes nothing on the server.

`GET /products/1`

POST

`@PostMapping`

Create something new from data you send.

`POST /products`

PUT

`@PutMapping`

Replace an existing item with what you send.

`PUT /products/1`

DELETE

`@DeleteMapping`

Remove an item from the collection.

`DELETE /products/1`

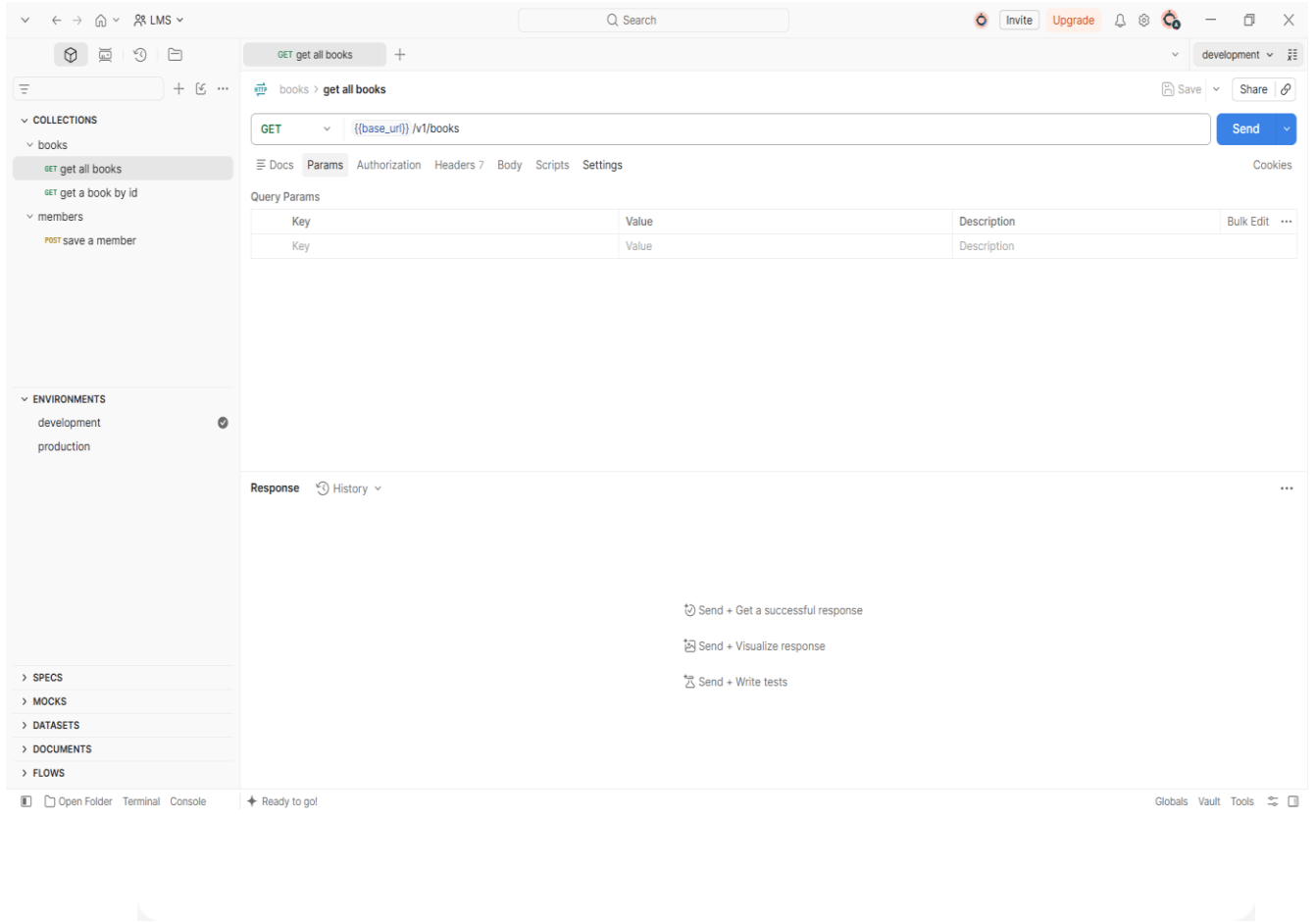
Every request is a verb plus a path. The verb says what to do; the path says what to do it to.

HTTP Request & Response Body (Payload)

HTTP Command	Request Has Body?	Response Has Body?	Primary Use Case Semantics
GET	**No**	**Yes**	Data retrieval
POST	**Yes**	**May Be**	Create new resource
PUT	**Yes**	**May Be**	Full resource replacement or creation if missing
PATCH	**Yes**	**May Be**	Partial field updates on existing resource
DELETE	**May**	**May Be**	Remove resource by identifier

Introduction to Postman

- **Workspace:** Dedicated space organizing team or individual API development and testing assets.
- **Collection:** Group of related API requests using plural nouns as best practice (e.g., products, members, etc).
- **Request:** Individual API call with method, URL, parameters, and payload.
- **Environment:** Set of key-value variables used to switch contexts dynamically (e.g., {{base_url}}).



Postman Collection: products

Environment Variable: base_url = **http://localhost:8080**

Action	Method & URL Pattern	Request Payload Body (JSON) / Notes
Create Product	**POST** {{base_url}}/products	Create a product {"name": "Keyboard", “price": 59.99}
Get All Products	**GET** {{base_url}}/products	No data in request payload body. Get all products from server side. (Spring Boot Application)
Get Product By ID	**GET** {{base_url}}/products/1	Retrieve single product by identifier via Path Variable
Delete Product By ID	**DELETE** {{base_url}}/products/1	Deletes product by identifier from backend server side via Path Variable

Test It - Postman Walkthrough

1. **Restart the application**

   Stop the running app and press Run again so the new endpoints load.

2. **POST /products with a JSON body**

   Send {"name": "Keyboard", "price": 59.99} and confirm a 200/201 response.

3. **GET /products**

   Get all the products from a Java ArrayList.

4. **GET /products/{id} and DELETE /products/{id}**

   Fetch a product by id and delete product by id.

References

- **Spring Framework Documentation: REST Web Services & Annotations**
  - **Source:** Spring Boot Reference Documentation
  - **Topics Covered:** @RestController, request mapping annotations, @PathVariable dynamic URI extraction, and @RequestBody HTTP payload binding.
  - **URL:** https://docs.spring.io/spring-framework/reference/web/webmvc.html
- **Baeldung Guide: Jackson & Spring Boot Integration**
  - **Source:** Baeldung Tutorials
  - **Topics Covered:** Practical implementation of Spring Boot REST controllers with Jackson, custom field serialization/deserialization, JavaBean naming conventions, and handling constructor binding.
  - **URL:** https://www.baeldung.com/jackson-object-mapper-tutorial
- **Jackson Documentation: Data Binding and Java Object Serialization**
  - **Source:** Jackson Databind Documentation
  - **Topics Covered:** ObjectMapper mechanics, getter-based property discovery for serialization, requirements for no-argument constructors during deserialization, and encapsulation handling.
  - **URL:** https://github.com/FasterXML/jackson-databind
- **REST Web API Design Best Practices Guide**
  - **Source:** Microsoft Azure Architecture Center
  - **Topics Covered:** Noun-based URI naming conventions, resource collection hierarchies, HTTP verb semantics (GET, POST, PUT, PATCH, DELETE), and payload structures.
  - **URL:** https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design
- **Postman Learning Center: Collections, Environments, and API Testing**
  - **Source:** Postman Official Documentation
  - **Topics Covered:** Configuring Postman Workspaces and Collections, environment variables (e.g., {{base_url}}), sending HTTP requests, and inspecting API responses.
  - **URL:** https://learning.postman.com/docs/use/send-requests/requests